ByteMart Digital

THE COMPLETE GUIDE

Hibernate

From Fundamentals to Professional Practice

A complete, example-driven guide to Hibernate — from ORM fundamentals, entity mapping, and associations to the persistence context, fetching, querying, caching, and tuning.

30	200+	14	8
Chapters	Code Examples	Visual Diagrams	Modules

© 2026 ByteMart Digital · All Rights Reserved

ByteMart Digital

Hibernate: The Complete Guide

First Edition · 2026

Copyright © 2026 ByteMart Digital. All rights reserved.
No part of this publication may be reproduced, distributed, or transmitted in any form or by any means — including photocopying, recording, screenshotting, or other electronic or mechanical methods — without the prior written permission of the publisher, except for brief quotations used in reviews and certain other non-commercial uses permitted by copyright law.

This product is licensed for personal use by the purchaser only. Sharing, reselling, or redistributing this file in whole or in part is strictly prohibited and constitutes a violation of copyright law.

Disclaimer
The information in this book is provided in good faith and for educational purposes. While every effort has been made to ensure accuracy, the author and publisher assume no responsibility for errors, omissions, or outcomes arising from the use of the code or information contained herein. All code examples are provided “as is”.

Trademarks
Hibernate is a trademark of Red Hat, Inc. All other product names, logos, and brands are the property of their respective owners and are used for identification purposes only. This guide is an independent educational work.

Published by ByteMart Digital

Web: bytemartdigital.com
Email: contact@bytemartdigital.in
Instagram: instagram.com/bytemartdigital.in
Telegram: t.me/bytemartcodingcommunity

Contents

1. What Is ORM & Hibernate? 4
2. Setup & Configuration 7
3. Entities & Identity 10
4. The Persistence Lifecycle 13
5. Basic Field Mapping 16
6. Embeddables & Value Types 19
7. Enums, Dates & Custom Types 22
8. Collections of Basic Types 25
9. One-to-One 28
10. One-to-Many & Many-to-One 31
11. Many-to-Many 34
12. Owning vs Inverse Side 37
13. Inheritance Strategies 40
14. Composite & Natural Keys 43
15. Advanced Mappings 46
16. The Session & Persistence Context 49
17. Flushing & Dirty Checking 52
18. Cascading & Orphan Removal 55
19. Detachment & Merging 58
20. Lazy vs Eager Loading 61
21. The N+1 Problem 64
22. Fetch Strategies 67
23. Pagination & Bulk Operations 70
24. JPQL & HQL 73
25. The Criteria API 76
26. Native Queries & Projections 79
27. Named Queries & DTO Projections 82
28. The Second-Level Cache 85
29. Transactions & Locking 88
30. Performance Tuning & Best Practices 91

MODULE 1 · ORM & HIBERNATE FOUNDATIONS

CHAPTER 01

What Is ORM & Hibernate?

The object-relational mismatch, how Hibernate bridges it, and how it relates to JPA.

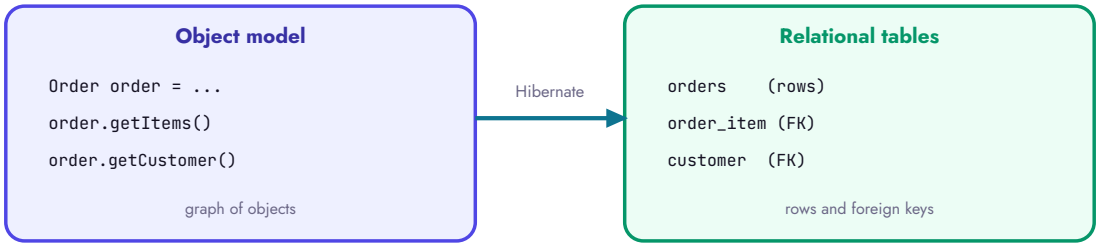
Objects and relational tables are fundamentally different shapes. **Object-Relational Mapping (ORM)** bridges that gap, and **Hibernate** is the most widely used ORM for Java — letting you persist an object graph to a database without writing the SQL to move data back and forth.

> **ORM**
> A technique that maps objects and their relationships to relational tables and foreign keys, so you work with objects while the framework handles the SQL.

The impedance mismatch

Objects have inheritance, references, and identity by memory address; tables have rows, foreign keys, and identity by primary key. This **object-relational impedance mismatch** is what an ORM exists to resolve.

ORM Bridges Objects and Tables



ORM maps the object graph to rows, resolving the impedance mismatch

Hibernate maps the object graph to rows and foreign keys, resolving the mismatch.

Hibernate and JPA

JPA (Jakarta Persistence API) is the specification; Hibernate is its most popular implementation. You mostly code to JPA's standard annotations and interfaces, and Hibernate provides the engine — plus powerful extras beyond the spec.

	JPA	Hibernate
Is a	specification	implementation
Provides	annotations, EntityManager	the ORM engine + extras
You use	standard API	JPA API (usually)

A first mapping

```java
@Entity
public class Product {
    @Id @GeneratedValue
    private Long id;
    private String name;
    private int price;
}
// no SQL: Hibernate generates INSERT, SELECT, UPDATE, DELETE
```

What Hibernate does for you

- Generates SQL for create, read, update, and delete.
- Maps rows to objects and back, including associations.
- Tracks changes and writes only what is dirty.
- Provides caching, lazy loading, and a query language (HQL).

Objects over SQL

Hibernate is an ORM that maps your object graph to relational tables, resolving the impedance mismatch. You code to JPA's standard API; Hibernate is the engine that turns objects into SQL and rows back into objects.

© 2026 ByteMart Digital

MODULE 1 · ORM & HIBERNATE FOUNDATIONS

CHAPTER 02

Setup & Configuration

The core Hibernate objects — the factory and the session — and how to configure them.

Hibernate is built around two objects: a **SessionFactory** (the JPA `EntityManagerFactory`), created once per application, and a **Session** (the `EntityManager`), a short-lived unit of work. Understanding these is the foundation for everything else.

Factory vs session

Object	Scope
SessionFactory	one per app, thread-safe, expensive
Session	one per unit of work, not thread-safe

Configuration

In a plain setup, configure the dialect, connection, and mapped classes. In Spring Boot, this is auto-configured from properties — you rarely touch it directly.

```text
# Spring Boot: the common properties
spring.datasource.url=jdbc:postgresql://localhost/shop
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.show-sql=true
```

Obtaining and using a session

```java
@PersistenceContext
private EntityManager em;   // the JPA session, injected by Spring

@Transactional
public void save(Product p) {
    em.persist(p);          // enrolls p in the persistence context
}
```

schema management (ddl-auto)

- `validate` — check the schema matches; make no changes (safe default).
- `update` — alter the schema to fit entities (dev convenience).
- `create-drop` — recreate on start, drop on stop (tests).
- `none` — do nothing; manage schema with migrations.

© 2026 ByteMart Digital

WATCH OUT

Never use `update` or `create-drop` in production. Manage the schema with a migration tool (Flyway or Liquibase) and set `ddl-auto=validate` so Hibernate only checks, never mutates, your database.

Factory once, session per unit of work

A SessionFactory is created once and shared; a Session is a short-lived, single-threaded unit of work. Configure Hibernate via properties, and in production let a migration tool own the schema while Hibernate merely validates it.

© 2026 ByteMart Digital

MODULE 1 · ORM & HIBERNATE FOUNDATIONS

CHAPTER 03

Entities & Identity

Marking classes as entities, defining the primary key, and choosing an ID generation strategy.

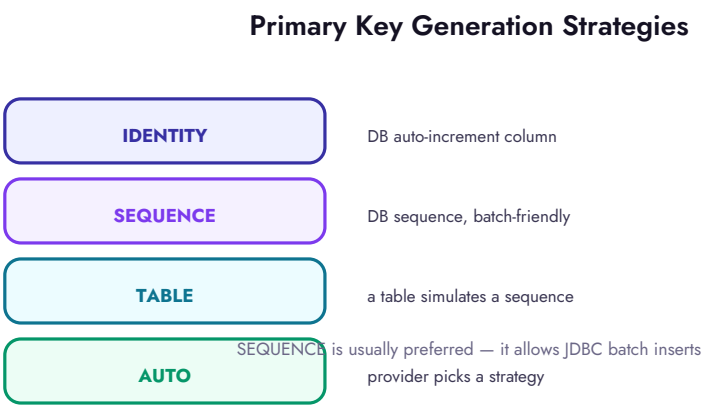
An **entity** is a class mapped to a table, and every entity needs an **identifier** — the field mapped to the primary key. How that identifier is generated has real consequences for performance, so it is worth understanding the strategies.

Defining an entity

Mark the class `@Entity` and one field `@Id`. Hibernate requires a no-argument constructor and a stable identifier.

```java
@Entity
@Table(name = "product")
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE)
    private Long id;
    private String name;
    protected Product() { }        // required no-arg constructor
}
```

ID generation strategies



IDENTITY, SEQUENCE, TABLE, and AUTO — the four key-generation strategies.

Strategy	How
IDENTITY	database auto-increment column
SEQUENCE	a database sequence object
TABLE	a table that simulates a sequence
AUTO	provider chooses per database

Why the strategy matters

`IDENTITY` forces Hibernate to insert each row immediately to learn its id, which **disables JDBC batch inserts**. `SEQUENCE` can pre-fetch ids and batch inserts, making it the better choice on databases that support sequences.

Natural vs surrogate keys

- A **surrogate key** (a generated number) is stable and simple — the usual choice.
- A **natural key** (like an ISBN) has business meaning but can change.
- Prefer surrogate keys for identity; enforce natural keys as unique constraints.
- The id should never change once assigned.

Every entity needs a stable id

An entity maps to a table and must have an identifier. Prefer a generated surrogate key, and favour `SEQUENCE` over `IDENTITY` where possible — it lets Hibernate batch inserts for much better write performance.

© 2026 ByteMart Digital

MODULE 1 · ORM & HIBERNATE FOUNDATIONS

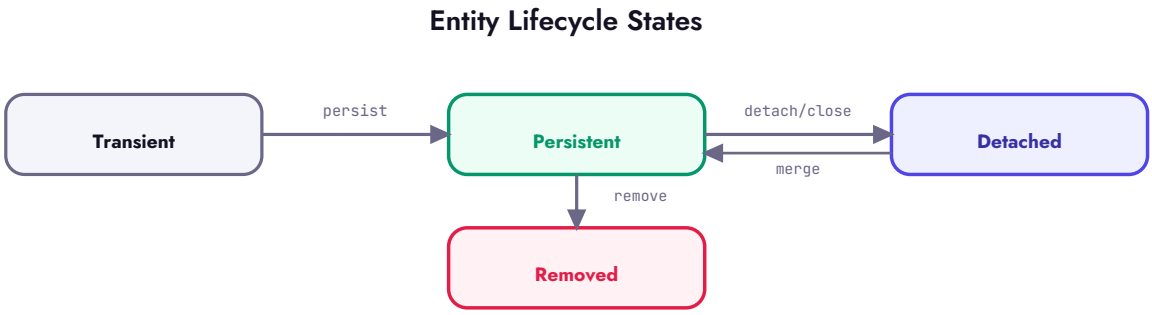
CHAPTER 04

The Persistence Lifecycle

The four entity states — transient, persistent, detached, removed — and the transitions between them.

An entity is not always 'in the database' or 'just an object'. Hibernate tracks it through four **lifecycle states**, and knowing which state an object is in explains when changes are saved, when they are not, and why the dreaded `LazyInitializationException` happens.

The four states



Transient, persistent, detached, and removed — and the operations that transition between them.

State	Meaning
Transient	new object, not yet managed
Persistent	managed by the session, tracked
Detached	was persistent, session closed
Removed	scheduled for deletion

Transitioning between states

java

```
Product p = new Product("Pen");   // TRANSIENT
em.persist(p);                    // -> PERSISTENT (tracked)
p.setName("Blue Pen");            // change auto-saved on flush
em.remove(p);                     // -> REMOVED (delete on flush)
// after the transaction/session ends -> DETACHED
```

Why the state matters

- Only **persistent** entities are auto-tracked — changes save without a call.
- Changes to a **transient** or **detached** object are **not** saved.
- Accessing a lazy field on a **detached** entity throws an exception.
- `merge` reattaches a detached entity's changes to a new session.

© 2026 ByteMart Digital

WATCH OUT

A very common bug: modifying a **detached** entity (after the transaction closed) and expecting it to save. It will not — only changes to persistent entities within an open session are flushed.

State decides what gets saved

Entities move through transient, persistent, detached, and removed states. Only persistent entities inside an open session are tracked and auto-saved — understanding this explains both silent 'lost' updates and lazy-loading exceptions.

© 2026 ByteMart Digital

MODULE 2 · ENTITY MAPPING

CHAPTER 05

Basic Field Mapping

Mapping fields to columns and controlling names, nullability, length, and precision.

By default Hibernate maps every field of an entity to a column, inferring the column name and type. `@Column` lets you take control — names, constraints, length, and precision — aligning the mapping with your schema.

Default mapping

Any non-transient field is persisted. The column name defaults to the field name, and the type to a sensible SQL type for the Java type.

java

```java
@Entity
public class Product {
    @Id @GeneratedValue Long id;
    private String name;        // -> column 'name' VARCHAR
    private int price;          // -> column 'price' INTEGER
    private boolean active;     // -> column 'active' BOOLEAN
}
```

Controlling the column

java

```java
@Column(name = "product_name", nullable = false, length = 120)
private String name;

@Column(precision = 12, scale = 2)   // for BigDecimal money
private BigDecimal price;

@Column(unique = true)
private String sku;
```

Common column controls

Attribute	Controls
name	the column name
nullable	NOT NULL constraint
length	VARCHAR length
unique	unique constraint
precision / scale	numeric precision

© 2026 ByteMart Digital

Excluding and sizing fields

- `@Transient` — a field that is not persisted at all.
- `@Lob` — map a large object (long text or binary).
- `@Basic(fetch = LAZY)` — load a big column only when accessed.
- Use `BigDecimal`, never `double`, for money.

TIP

Let the schema, not Hibernate, be the source of truth in production: define columns and constraints in migrations, and use `@Column` mainly to match them so `ddl-auto=validate` passes.

Fields become columns, precisely

Hibernate maps fields to columns automatically; `@Column` lets you control names, constraints, and sizes to match your schema. Use the right types — `BigDecimal` for money — and mark non-persistent fields `@Transient`.

© 2026 ByteMart Digital

MODULE 2 · ENTITY MAPPING

CHAPTER 06

Embeddables & Value Types

Grouping related fields into reusable value objects that map into the owner's table.

Not everything deserves its own table. An **embeddable** is a value object — like an address or a money amount — whose fields are stored as columns in the owning entity's table. It gives you clean object structure without an extra join.

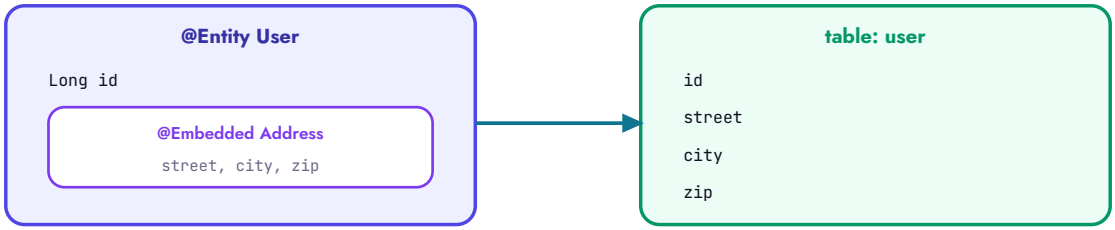
Defining an embeddable

java

```java
@Embeddable
public class Address {
    private String street;
    private String city;
    private String zip;
}

@Entity
public class Customer {
    @Id @GeneratedValue Long id;
    @Embedded
    private Address address;      // fields become columns of 'customer'
}
```

Embeddables Flatten Into the Table



A value object's fields become columns in the owner's table — no join

An embeddable's fields become columns in the owning entity's table — no join.

Value types vs entities

	Entity	Value type
Has identity	yes (@Id)	no
Own table	yes	no (owner's table)
Shared	referenced	owned, not shared

Reusing an embeddable twice

To embed the same type twice, override the column names with `@AttributeOverride` so the two copies do not clash.

```java
@Embedded
@AttributeOverride(name = "street", column = @Column(name = "bill_street"))
private Address billing;

@Embedded
@AttributeOverride(name = "street", column = @Column(name = "ship_street"))
private Address shipping;
```

Why use embeddables

- Group related fields into a meaningful **value object**.
- Reuse the same structure across entities.
- Keep everything in one table — no join, good performance.
- Model concepts with **no identity of their own** (address, money, range).

Structure without a table

Embeddables let you model value objects — addresses, money, ranges — that have no identity, storing their fields in the owner's table. You get clean object design and reuse with zero extra joins.

MODULE 2 · ENTITY MAPPING

CHAPTER 07

Enums, Dates & Custom Types

Mapping enums safely, handling modern date/time types, and converting custom types.

Beyond strings and numbers, real entities hold enums, dates, and domain-specific types. Mapping each correctly — especially enums and dates — avoids some classic bugs, and **attribute converters** let you map any type you like.

Enums — map by name, not ordinal

Always map enums with `EnumType.STRING`. The default, `ORDINAL`, stores the position — so reordering or inserting an enum value silently corrupts existing data.

```java
@Enumerated(EnumType.STRING)     // stores 'ACTIVE', not 0
private Status status;

// NEVER rely on the default ORDINAL for a stored enum
```

WATCH OUT

`EnumType.ORDINAL` is a data-corruption trap: it stores the enum's index, so adding or reordering constants remaps every existing row's meaning. Always use `EnumType.STRING`.

Date and time

Use the modern `java.time` types — `LocalDate`, `LocalDateTime`, `Instant` — which Hibernate maps directly and correctly. Avoid the legacy `java.util.Date`.

```java
private LocalDate orderDate;         // DATE
private LocalDateTime createdAt;     // TIMESTAMP
private Instant processedAt;         // TIMESTAMP (UTC instant)
```

Attribute converters

An `AttributeConverter` maps a custom type to a database column and back — perfect for value objects, encrypted fields, or storing a type as JSON or a delimited string.

```java
@Converter(autoApply = true)
public class MoneyConverter implements AttributeConverter<Money, Long> {
    public Long convertToDatabaseColumn(Money m) { return m.cents(); }
    public Money convertToEntityAttribute(Long c) { return new Money(c); }
}
```

© 2026 ByteMart Digital

Map special types deliberately

Store enums as strings, use `java.time` for dates, and reach for an `AttributeConverter` to map any custom type. These small choices prevent data-corruption bugs and keep your domain types clean.

© 2026 ByteMart Digital

MODULE 2 · ENTITY MAPPING

CHAPTER 08

Collections of Basic Types

Storing lists, sets, and maps of simple values with @ElementCollection.

Sometimes an entity owns a collection of simple values — tags, phone numbers, scores — that are not entities in their own right. `@ElementCollection` maps these to a separate table owned entirely by the parent.

A collection of values

```java
@Entity
public class Article {
    @Id @GeneratedValue Long id;

    @ElementCollection
    @CollectionTable(name = "article_tag",
        joinColumns = @JoinColumn(name = "article_id"))
    @Column(name = "tag")
    private Set<String> tags = new HashSet<>();
}
```

Element collections vs associations

- Elements are **value types**, not entities — no identity, no shared references.
- They are fully **owned** by the parent and deleted with it.
- The collection table has a foreign key back to the parent.
- Use it for tags, settings, or embeddable value collections.

Collections of embeddables

The elements can be embeddables, not just primitives — storing structured value objects in the collection table.

```java
@ElementCollection
private List<Address> previousAddresses = new ArrayList<>();
```

When to use an entity instead

WATCH OUT

If the elements have their own identity, need to be shared, or are queried independently, they should be a proper entity with an association — not an element collection. Element collections are for owned, dependent values only.

© 2026 ByteMart Digital

Owned collections of values

`@ElementCollection` stores collections of primitives or embeddables in a child table wholly owned by the parent. Reach for it for tags and settings — but use a real entity association whenever the elements have identity or are shared.

© 2026 ByteMart Digital

MODULE 3 · ASSOCIATIONS

CHAPTER 09

One-to-One

Mapping a one-to-one relationship, and the ways to place and share the foreign key.

A **one-to-one** association links exactly one entity to one other — a user and their profile, an order and its invoice. It is the simplest association, but where the foreign key lives, and whether to load it lazily, both need thought.

A foreign-key one-to-one

The common form: one side holds a foreign key to the other. Put `@OneToOne` and `@JoinColumn` on the owning side.

```java
@Entity
public class User {
    @Id @GeneratedValue Long id;
    @OneToOne
    @JoinColumn(name = "profile_id")
    private Profile profile;         // owning side, holds the FK
}
```

Making it bidirectional

To navigate both ways, add the inverse side with `mappedBy`, which points to the field that owns the relationship.

```java
@Entity
public class Profile {
    @Id @GeneratedValue Long id;
    @OneToOne(mappedBy = "profile")
    private User user;               // inverse side, no FK
}
```

Shared primary key

Alternatively, the two rows can share the same primary key with `@MapsId` — no separate foreign-key column, and a guaranteed one-to-one at the database level.

```java
@OneToOne
@MapsId
private User user;   // profile.id == user.id
```

WATCH OUT

A lazy `@OneToOne` on the inverse (non-owning) side often loads eagerly anyway, because Hibernate must check whether the other row exists. If lazy loading matters, prefer a foreign key on the side you query from.

One row, one row — mind the key

A one-to-one links two entities exclusively. Decide which side owns the foreign key (or share the primary key with `@MapsId`), and remember that lazy loading on the inverse side is unreliable.

MODULE 3 · ASSOCIATIONS

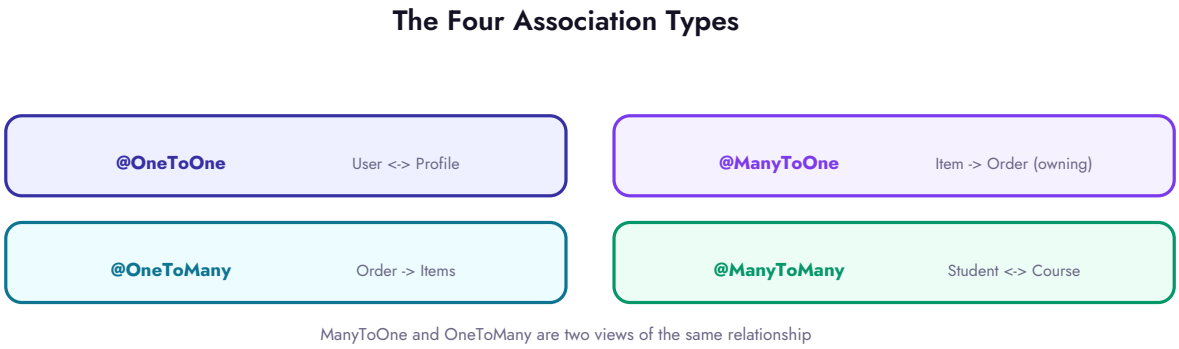
CHAPTER 10

One-to-Many & Many-to-One

The most common association — a parent with many children — and how to map it well.

The **one-to-many** / **many-to-one** pair is the workhorse of ORM: an order with many items, a department with many employees. They are two views of the same relationship, and mapping them correctly — with the foreign key on the many side — is essential.

The four association types



OneToOne, ManyToOne, OneToMany, ManyToMany — the four relationship shapes.

Many-to-one (the owning side)

The **many** side holds the foreign key and owns the relationship. This is the side you almost always map first.

```java
@Entity
public class OrderItem {
    @Id @GeneratedValue Long id;
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "order_id")
    private Order order;             // owning side, holds order_id
}
```

One-to-many (the inverse side)

The **one** side maps the collection with `mappedBy`, pointing at the field on the many side. It does not own the foreign key.

```java
@Entity
public class Order {
    @Id @GeneratedValue Long id;
    @OneToMany(mappedBy = "order", cascade = CascadeType.ALL,
               orphanRemoval = true)
    private List<OrderItem> items = new ArrayList<>();
}
```

Keep both sides in sync

In a bidirectional relationship, always set both ends together — a helper method prevents inconsistent object graphs.

```java
public void addItem(OrderItem item) {
    items.add(item);
    item.setOrder(this);   // keep both sides consistent
}
```

WATCH OUT

Always make `@ManyToOne` associations `LAZY` — they are **eager by default**, which quietly loads the parent on every child fetch and fuels the N+1 problem covered later.

Foreign key on the many side

The one-to-many/many-to-one pair models parent-child data. Map the foreign key on the many side as the owner, use `mappedBy` on the collection, make it lazy, and keep both sides synchronised with helper methods.

© 2026 ByteMart Digital

MODULE 3 · ASSOCIATIONS

CHAPTER 11

Many-to-Many

Mapping many-to-many relationships through a join table — and when to model the join explicitly.

A **many-to-many** relationship — students and courses, users and roles — connects many on each side through a **join table**. Hibernate can manage that table for you, but as soon as the relationship needs its own data, you should model it as an entity.

A join-table many-to-many

java

```
@Entity
public class Student {
    @Id @GeneratedValue Long id;
    @ManyToMany
    @JoinTable(name = "enrolment",
        joinColumns = @JoinColumn(name = "student_id"),
        inverseJoinColumns = @JoinColumn(name = "course_id"))
    private Set<Course> courses = new HashSet<>();
}
```

Use Set, not List

Map many-to-many collections as a `Set`. With a `List`, Hibernate may delete and re-insert all join rows on a single change — inefficient and surprising.

When to promote the join to an entity

The moment the relationship needs its own attributes — an enrolment date, a grade, a quantity — the join table becomes a real entity with two many-to-one associations.

java

```
@Entity
public class Enrolment {
    @Id @GeneratedValue Long id;
    @ManyToOne Student student;
    @ManyToOne Course course;
    private LocalDate enrolledOn;   // the extra data the join needs
    private String grade;
}
```

Guidance

- Plain `@ManyToMany` only when the join has **no** extra data.
- Map collections as `Set` for efficient updates.
- Promote to an **association entity** as soon as the join needs attributes.
- Keep both sides synchronised, as with any bidirectional link.

© 2026 ByteMart Digital

Join table, or join entity

Many-to-many links are mapped through a join table with `@ManyToMany` and a `Set`. But the instant the relationship carries its own data, model the join as an entity with two many-to-one associations — it is almost always the right call.

© 2026 ByteMart Digital

MODULE 3 · ASSOCIATIONS

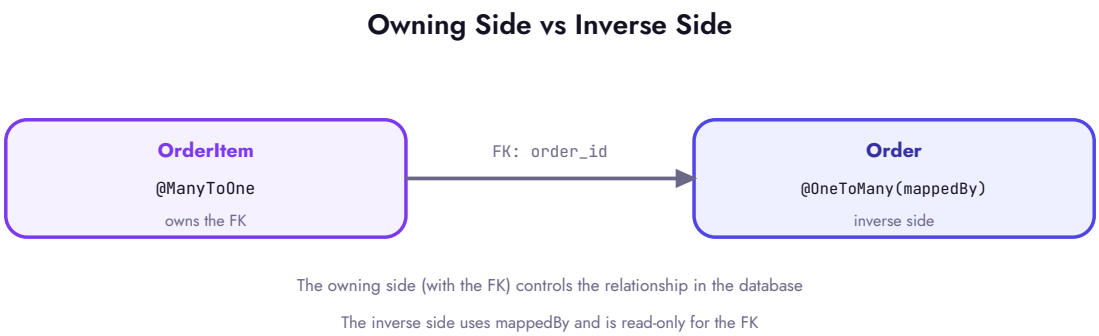
CHAPTER 12

Owning vs Inverse Side

The single concept that makes bidirectional associations work — who owns the foreign key.

The most misunderstood idea in Hibernate associations is the **owning side**. In a bidirectional relationship, only one side controls the foreign key in the database — and if you update the wrong side, your change is silently ignored. Master this and associations stop being mysterious.

Owning vs inverse



The owning side holds the foreign key; the inverse side uses mappedBy and is read-only for it.

- The **owning side** holds the foreign key and controls the relationship.
- The **inverse side** declares `mappedBy` and does not own the FK.
- Hibernate persists the relationship based on the **owning** side only.
- In many-to-one/one-to-many, the **many** side always owns.

The classic bug

Add a child to the parent's collection (the inverse side) but forget to set the child's reference to the parent (the owning side), and the foreign key is never written — the change vanishes.

```java
// BUG: only the inverse side is updated
order.getItems().add(item);          // inverse side
// item.setOrder(order) is missing -> FK never set

// CORRECT: set the owning side (a helper does both)
order.addItem(item);                 // sets item.order internally
```

Why it works this way

A single foreign-key column can be controlled by only one side. Hibernate reads that side to decide the column's value; the other side exists purely for convenient navigation in your object model.

TIP

Always write **helper methods** (`addItem`, `removeItem`) that update both ends. They make it impossible to forget the owning side and keep your in-memory object graph consistent with the database.

One side owns the foreign key

In every bidirectional association, exactly one side owns the foreign key; updates to the inverse side alone are ignored. Know which side owns it, always update that side, and use helper methods to keep both consistent.

MODULE 4 · INHERITANCE & ADVANCED MAPPING

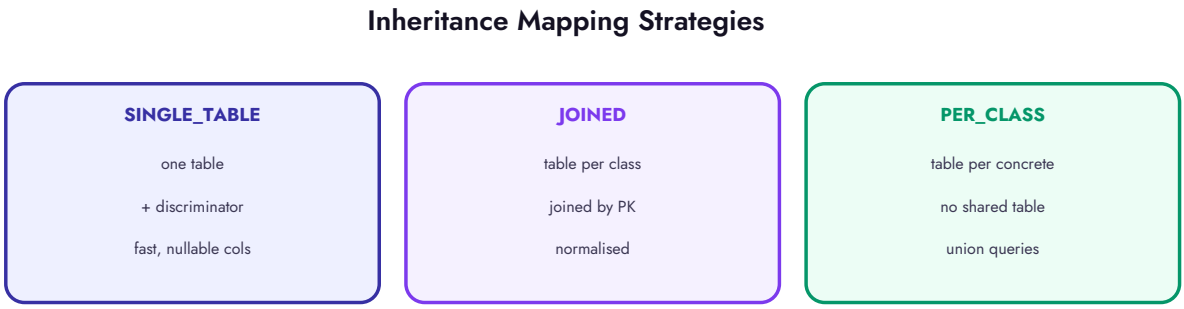
CHAPTER 13

Inheritance Strategies

The three ways to map a class hierarchy to tables, and the trade-offs of each.

Objects have inheritance; tables do not. Hibernate offers three strategies to map a class hierarchy to relational tables, each trading normalisation against query performance. Choosing the right one for your hierarchy is an important design decision.

The three strategies



SINGLE_TABLE, JOINED, and TABLE_PER_CLASS — three ways to map a hierarchy.

Single table

All classes in the hierarchy share one table, with a **discriminator column** marking each row's type. It is the fastest — no joins — but subclass columns must be nullable.

```java
@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "payment_type")
public abstract class Payment { @Id @GeneratedValue Long id; }

@Entity @DiscriminatorValue("CARD")
public class CardPayment extends Payment { String cardLast4; }
```

Comparing the strategies

Strategy	Trade-off
SINGLE_TABLE	fastest; nullable subclass columns
JOINED	normalised; joins on read
TABLE_PER_CLASS	no shared table; union queries

Joined

Each class gets its own table, linked by primary key. It is fully normalised with no nullable columns, but reading a subclass requires a join back to the parent table.

Choosing

- **SINGLE_TABLE** — default choice; best read performance.
- **JOINED** — when normalisation and constraints matter most.
- **TABLE_PER_CLASS** — rarely; polymorphic queries are costly.
- Prefer **composition over inheritance** when the hierarchy is shaky.

Map hierarchies by query needs

Three strategies map inheritance: SINGLE_TABLE (fast, denormalised), JOINED (normalised, joins on read), and TABLE_PER_CLASS (rare). Default to SINGLE_TABLE, choose JOINED when constraints matter, and consider composition instead.

MODULE 4 · INHERITANCE & ADVANCED MAPPING

CHAPTER 14

Composite & Natural Keys

Mapping primary keys made of several columns, and enforcing business keys.

Most entities use a single generated id, but some need a **composite key** — a primary key made of several columns — or must enforce a **natural key** that has business meaning. Hibernate supports both, though they add complexity worth minimising.

Composite keys with @EmbeddedId

Model the composite key as an embeddable and mark it `@EmbeddedId`. The key class must implement `equals` and `hashCode` over all its fields.

```java
@Embeddable
public record OrderLineId(Long orderId, int lineNo) { }

@Entity
public class OrderLine {
    @EmbeddedId
    private OrderLineId id;
    private int quantity;
}
```

The alternative: @IdClass

`@IdClass` achieves the same with separate id fields on the entity plus a matching id class. `@EmbeddedId` is usually cleaner and preferred.

Natural keys

A **natural key** — an ISBN, a country code — carries business meaning. Even with a surrogate primary key, enforce the natural key as a unique constraint and, optionally, mark it `@NaturalId` for Hibernate-optimised lookups.

```java
@Entity
public class Book {
    @Id @GeneratedValue Long id;      // surrogate key
    @NaturalId
    @Column(unique = true) String isbn;   // business key
}
```

Guidance

- Prefer a **surrogate** primary key; enforce natural keys as unique.
- Reserve composite keys for join tables and genuinely composite identities.

© 2026 ByteMart Digital

- A composite-key class must have correct `equals/hashCode`.
- `@NaturalId` enables an efficient, cacheable lookup by business key.

Composite when you must, surrogate by default

Hibernate maps composite keys with `@EmbeddedId` (or `@IdClass`) and supports business keys via `@NaturalId`. Prefer a simple surrogate key, and add composite or natural keys only where the data genuinely demands them.

MODULE 4 · INHERITANCE & ADVANCED MAPPING

CHAPTER 15

Advanced Mappings

Secondary tables, derived properties, and mapping columns that do not map one-to-one.

Real schemas are not always tidy. Sometimes an entity's data spans two tables, a property is computed by the database, or a value is read-only. Hibernate has focused tools — secondary tables, formulas, and generated values — for these less-common but real situations.

Secondary tables

`@SecondaryTable` maps one entity across two tables joined by primary key — useful when a legacy schema splits an entity's columns.

```java
@Entity
@Table(name = "product")
@SecondaryTable(name = "product_detail")
public class Product {
    @Id @GeneratedValue Long id;
    String name;                              // in 'product'
    @Column(table = "product_detail")
    String longDescription;                   // in 'product_detail'
}
```

Derived / computed properties

`@Formula` maps a read-only property to a SQL expression evaluated on load — handy for computed values you do not want to store.

```java
@Formula("(select count(*) from order_item i where i.order_id = id)")
private int itemCount;   // computed by the database, read-only
```

Generated and default values

- `@GeneratedValue` — the database or provider assigns it.
- `@ColumnDefault` — supply a schema default value.
- `@Generated` — a value computed by the database on insert/update.
- `@CreationTimestamp` / `@UpdateTimestamp` — auto-managed audit times.

```java
@CreationTimestamp private Instant createdAt;
@UpdateTimestamp   private Instant updatedAt;   // set automatically
```

WATCH OUT

Use these advanced mappings sparingly. They are the right tool for legacy schemas and genuine edge cases, but a design that needs many of them is often a sign the schema and object model have drifted apart.

Tools for awkward schemas

Secondary tables, `@Formula`, and generated values handle entities that span tables, computed properties, and auto-managed columns. Keep them for legacy and edge cases — a clean schema rarely needs many of them.

MODULE 5 · THE PERSISTENCE CONTEXT

CHAPTER 16

The Session & Persistence Context

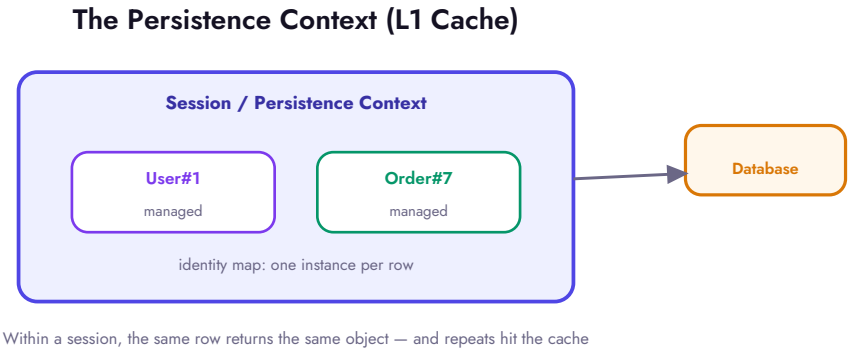
The first-level cache, the identity map, and why the persistence context is central to Hibernate.

The **persistence context** — the session's first-level cache — is the beating heart of Hibernate. It tracks every managed entity, guarantees one object per row, and is what makes automatic change tracking and caching possible. Almost every Hibernate behaviour flows from it.

> **Persistence context**
> A session-scoped cache that holds all managed entities, ensuring a single instance per database row and tracking their changes for automatic synchronisation.

The identity map

Within one session, loading the same row twice returns the **same object**. The persistence context maintains an identity map — one instance per primary key — so == works and there is no duplication.



The session holds one managed instance per row; repeat loads hit this first-level cache.

```java
Product a = em.find(Product.class, 1L);
Product b = em.find(Product.class, 1L);
System.out.println(a == b);   // true — same instance, second load is cached
```

The first-level cache

This cache is always on and cannot be disabled. A repeated `find` for an already-loaded entity hits the context, not the database — saving a query within the unit of work.

Scope and lifecycle

- The context lives for the duration of the **session / transaction**.
- It holds every entity you load or persist as **managed**.
- It is **not** shared between sessions or threads.
- Closing the session clears it — its entities become detached.

WATCH OUT

The first-level cache can grow large in a long transaction that loads many entities, consuming memory. For bulk processing, clear the context periodically with `em.clear()` or use a stateless session.

One context, one instance per row

The persistence context is a session-scoped first-level cache and identity map: one managed object per row, change tracking, and cached repeat loads. Understanding it is understanding how Hibernate actually works.

© 2026 ByteMart Digital

MODULE 5 · THE PERSISTENCE CONTEXT

CHAPTER 17

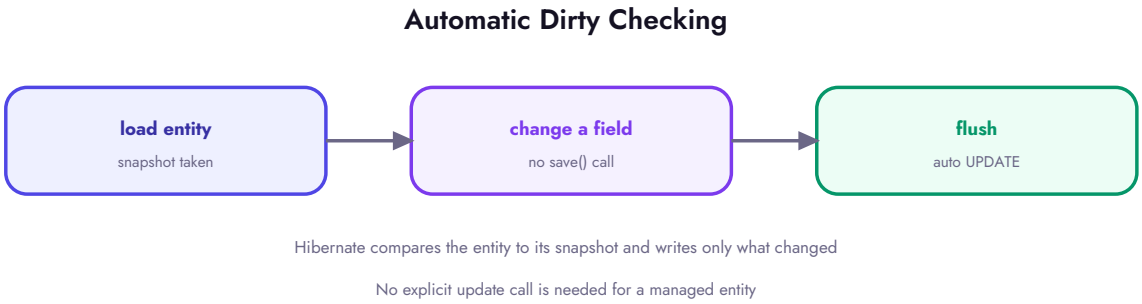
Flushing & Dirty Checking

How Hibernate detects changes automatically and decides when to write them to the database.

One of Hibernate's most surprising conveniences: you change a managed entity's field and never call `save`, yet the change is written to the database. This is **automatic dirty checking**, and **flushing** is the moment those changes are synchronised.

Dirty checking

When it loads an entity, Hibernate takes a snapshot. At flush time it compares the current state to that snapshot and generates an `UPDATE` for exactly the fields that changed — no explicit call needed.



Hibernate snapshots on load, then flush compares and writes only changed fields.

```java
@Transactional
public void rename(Long id, String name) {
    Product p = em.find(Product.class, id);   // snapshot taken
    p.setName(name);                           // no save() call
}   // on commit, Hibernate flushes an UPDATE automatically
```

When flushing happens

- Automatically before a **transaction commits**.
- Before a query that might be affected by pending changes.
- Explicitly, when you call `em.flush()`.
- Flush writes SQL; the transaction commit makes it permanent.

Flush is not commit

Flushing sends the SQL to the database; **committing** makes it durable. A flush without a commit can still be rolled back — the two are distinct steps.

Controlling flush timing

WATCH OUT

Do not call `em.flush()` needlessly in a loop — it forces SQL early and can hurt performance. Let Hibernate flush at commit unless you specifically need writes visible to a subsequent query in the same transaction.

Change tracked, written on flush

Hibernate snapshots managed entities and, at flush time, writes only what changed — no `save` call required. Flushing sends the SQL; commit makes it durable. This automatic dirty checking is why Hibernate feels almost magical.

© 2026 ByteMart Digital

MODULE 5 · THE PERSISTENCE CONTEXT

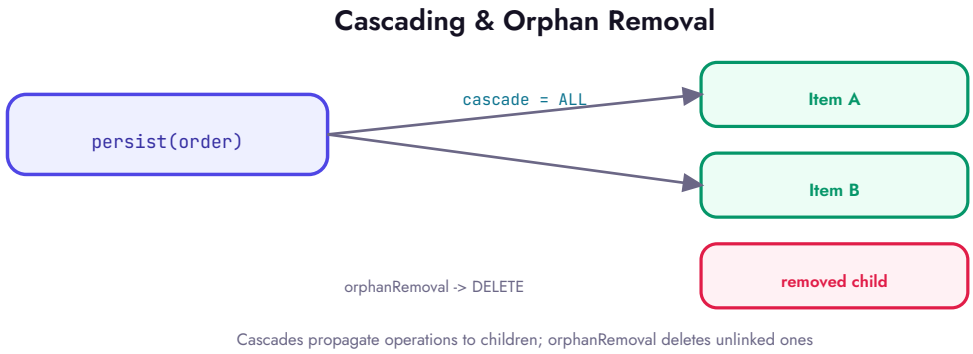
CHAPTER 18

Cascading & Orphan Removal

Propagating persistence operations to associated entities and cleaning up orphaned children.

By default, persisting a parent does nothing to its children. **Cascading** propagates operations — persist, merge, remove — across an association, and **orphan removal** deletes children that are unlinked from their parent. Together they let you manage an object graph as one unit.

Cascade types



Cascades propagate operations to children; orphanRemoval deletes unlinked children.

Cascade	Propagates
PERSIST	saving the parent saves children
MERGE	merging the parent merges children
REMOVE	deleting the parent deletes children
ALL	all of the above

Applying cascade

```java
@OneToMany(mappedBy = "order",
           cascade = CascadeType.ALL,
           orphanRemoval = true)
private List<OrderItem> items = new ArrayList<>();

// now persisting the order persists its items automatically
em.persist(order);   // items saved too
```

Orphan removal

`orphanRemoval = true` deletes a child when it is removed from the parent's collection — modelling a strict parent-owns-child relationship where a child cannot exist alone.

```java
order.getItems().remove(item);   // with orphanRemoval, this DELETEs it
```

Use cascades carefully

WATCH OUT

Cascade `REMOVE` and `orphanRemoval` only where the child genuinely belongs to the parent (an order and its lines). Applied to shared entities — like a category referenced by many products — they can delete data other entities still need.

Manage the graph as a unit

Cascading propagates persist, merge, and remove across associations, and orphan removal deletes unlinked children — letting you treat a parent and its owned children as one aggregate. Apply them only to true parent-owns-child relationships.

MODULE 5 · THE PERSISTENCE CONTEXT

CHAPTER 19

Detachment & Merging

Working with entities after the session closes, and reattaching their changes with merge.

Once a session closes, its entities become **detached** — no longer tracked, their changes no longer saved automatically. This is common in web applications, where an entity is loaded, sent to a browser, and returned later. `merge` is how you bring those changes back.

Detached entities

A detached entity is a plain object holding data that was once managed. Modifying it does nothing to the database until you reattach it to a session.

```java
Product p;
tx1: { p = em.find(Product.class, 1L); }   // p is now DETACHED
p.setName("Updated");                        // change not saved
// need a new session and merge to persist it
```

Merging

`merge` copies a detached entity's state onto a managed instance in the current session and returns that managed instance. Crucially, it returns a new object — the argument stays detached.

```java
@Transactional
public Product update(Product detached) {
    Product managed = em.merge(detached);   // copies state in
    return managed;      // use the returned instance, not the argument
}
```

WATCH OUT

A classic mistake: calling `merge(entity)` and then continuing to use the original `entity` variable, which is still detached. Always work with the **returned** instance — that is the managed one.

Detached data and lazy fields

- Accessing an **unloaded lazy** field on a detached entity throws.
- Load everything you need **before** the session closes.
- `merge` reattaches state; it does not require the entity to still be managed.
- For long conversations, keep the working data on detached entities and merge at the end.

© 2026 ByteMart Digital

Reattach with the returned instance

When a session closes, entities detach and stop tracking changes; `merge` copies their state back into a new managed instance — which you must then use in place of the original. This is the standard pattern for edit-then-save web flows.

© 2026 ByteMart Digital

MODULE 6 · FETCHING & PERFORMANCE

CHAPTER 20

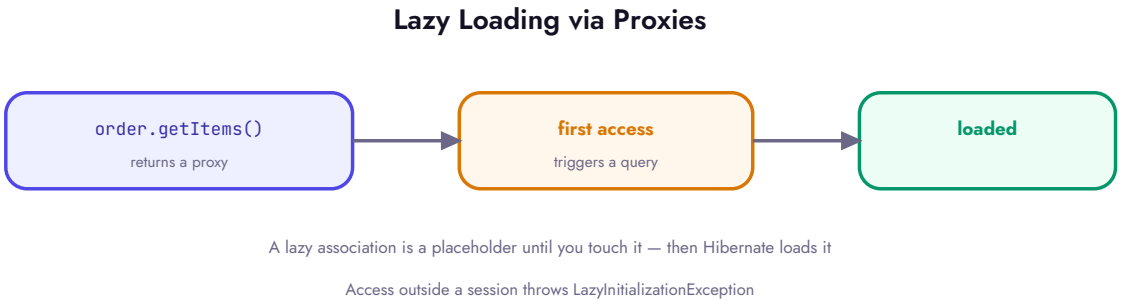
Lazy vs Eager Loading

How Hibernate defers loading associations with proxies, and when each strategy applies.

Loading an entity does not have to load its whole object graph. **Lazy loading** defers an association until you actually access it, using a proxy; **eager loading** fetches it upfront. Choosing well is one of the biggest levers on Hibernate performance.

How lazy loading works

A lazy association returns a **proxy** — a lightweight placeholder. The real data is fetched with a separate query the first time you touch the proxy.



A lazy association is a proxy until first access, which triggers the load query.

The defaults

Association	Default fetch
@ManyToOne	EAGER (change to LAZY)
@OneToOne	EAGER (change to LAZY)
@OneToMany	LAZY (good)
@ManyToMany	LAZY (good)

WATCH OUT

The `@ManyToOne` and `@OneToOne` defaults are **EAGER** — a frequent performance trap. Make them `LAZY` explicitly, then fetch what you need per query.

LazyInitializationException

Accessing a lazy association **after** the session has closed throws `LazyInitializationException` — the proxy has no session to load from. The fix is to fetch the data while the session is open, not to make everything eager.

```java
Order o;
tx: { o = em.find(Order.class, 1L); }   // session closes here
o.getItems().size();   // throws: no session to load the proxy
```

The right default

- Make **everything lazy** by default.
- Fetch exactly what each use case needs with a query.
- Never make an association eager to 'fix' a lazy exception.
- Load the graph you need inside the transaction.

Lazy by default, fetch on purpose

Lazy loading defers associations behind proxies; eager loads them upfront. Make everything lazy — especially the eager-by-default `@ManyToOne` — and fetch precisely what each query needs, rather than reacting to lazy exceptions with eager mappings.

© 2026 ByteMart Digital

MODULE 6 · FETCHING & PERFORMANCE

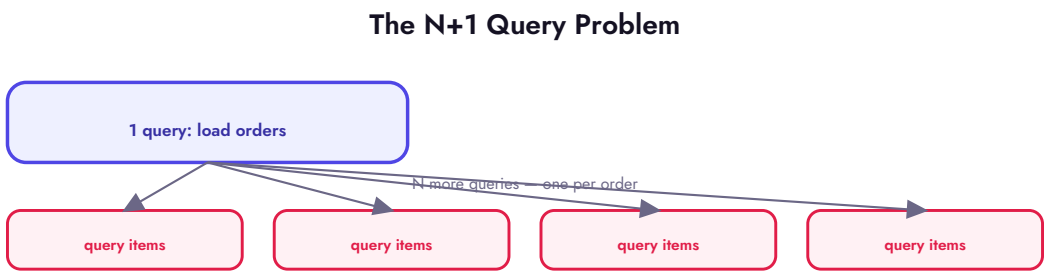
CHAPTER 21

The N+1 Problem

The most common Hibernate performance bug — how it happens and how to eliminate it.

The **N+1 select problem** is the single most common Hibernate performance issue. You run one query to load a list, then — as you access each element's lazy association — Hibernate fires one more query per element. A list of 100 orders can quietly become 101 queries.

How it happens



One query loads the parents; touching each lazy collection fires N more queries.

```java
List<Order> orders = em.createQuery("from Order", Order.class)
                       .getResultList();          // 1 query
for (Order o : orders) {
    o.getItems().size();                            // +1 query each
}   // total: 1 + N queries
```

Why it hides

It works correctly and looks innocent — the extra queries are invisible in the code. It only surfaces as slowness under real data volumes, which is why enabling SQL logging in development is so valuable.

The fixes

- **Fetch join** — load the association in the same query.
- **Entity graph** — declare what to fetch, reusably.
- **Batch fetching** — load lazy associations in batches, not one by one.
- The next chapter covers each of these in detail.

© 2026 ByteMart Digital

java

```java
// Fix with a fetch join: one query loads orders AND items
em.createQuery(
  "select distinct o from Order o join fetch o.items", Order.class)
 .getResultList();
```

TIP

Turn on SQL logging (`show-sql` plus statistics) during development. The N+1 problem is obvious the moment you see a burst of near-identical queries — and nearly invisible until then.

One plus N, hidden until it hurts

The N+1 problem fires one query per element of a loaded list as you touch lazy associations — correct but slow at scale. Detect it with SQL logging and fix it with a fetch join, entity graph, or batch fetching.

© 2026 ByteMart Digital

MODULE 6 · FETCHING & PERFORMANCE

CHAPTER 22

Fetch Strategies

Loading exactly the data you need with join fetch, entity graphs, and batch fetching.

The cure for N+1 and over-fetching is to load precisely the right data per use case. Hibernate gives you three complementary tools: **fetch joins** in queries, reusable **entity graphs**, and **batch fetching** to reduce query counts.

Join fetch

A `join fetch` in a query loads an association in the same SQL statement — the most direct fix for N+1 when you know you need the association.

java

```
List<Order> orders = em.createQuery("""
    select distinct o from Order o
    join fetch o.items
    join fetch o.customer
    """, Order.class).getResultList();   // one query, fully loaded
```

Entity graphs

An **entity graph** declares which associations to fetch, applied to a query or find as a hint — reusable and separate from the query text.

java

```
@EntityGraph(attributePaths = {"items", "customer"})
List<Order> findByStatus(Status status);   // Spring Data repository
```

Batch fetching

`@BatchSize` tells Hibernate to load lazy associations in batches — turning N+1 into a handful of `IN` queries instead of N single-row queries.

java

```
@OneToMany(mappedBy = "order")
@BatchSize(size = 20)   // load items for up to 20 orders per query
private List<OrderItem> items;
```

Choosing a strategy

- **Join fetch** — when a specific query always needs the association.
- **Entity graph** — to declare fetch plans reusably, outside the query.
- **Batch fetching** — a good global default that softens N+1 everywhere.

© 2026 ByteMart Digital

- Avoid multiple **collection** join fetches in one query (a cartesian product).

WATCH OUT

Do not `join fetch` two collections in one query — it produces a cartesian product that multiplies rows. Fetch one collection per query, or use batch fetching / entity graphs instead.

Fetch precisely, per use case

Load exactly what each operation needs: a fetch join for known associations, an entity graph for reusable plans, and batch fetching as a safety net. Precise fetching is how you kill N+1 without over-fetching everything eagerly.

© 2026 ByteMart Digital

MODULE 6 · FETCHING & PERFORMANCE

CHAPTER 23

Pagination & Bulk Operations

Handling large result sets with paging, streaming, and set-based bulk updates.

Loading thousands of entities into the persistence context is slow and memory-hungry. For large data sets you need **pagination**, streaming, or set-based **bulk operations** that bypass the per-entity machinery entirely.

Pagination

Fetch results in pages with an offset and limit rather than all at once. Spring Data's `Pageable` wraps this cleanly.

```java
List<Product> page = em.createQuery("from Product", Product.class)
    .setFirstResult(page * size)   // offset
    .setMaxResults(size)           // limit
    .getResultList();
```

WATCH OUT

Do not paginate a query that uses a collection `join fetch` — the row multiplication breaks offset/limit, forcing Hibernate to paginate in memory over the whole result. Fetch the collection separately, or use batch fetching.

Bulk update and delete

To change many rows, a single set-based `UPDATE` or `DELETE` is far faster than loading and modifying each entity. These run directly in the database.

```java
int updated = em.createQuery(
    "update Product p set p.active = false where p.stock = 0")
    .executeUpdate();   // one statement, no entities loaded
```

Bulk operations bypass the context

- Bulk statements do **not** update the persistence context or caches.
- They ignore cascades and optimistic-lock versions.
- Clear the context afterward if you continue in the same session.
- Use them for large data changes; use entities for normal, small changes.

Processing huge data sets

For very large reads, stream results and clear the context periodically — or use a **stateless session**, which has no persistence context and no dirty checking, keeping memory flat.

© 2026 ByteMart Digital

Scale reads and writes deliberately

Page or stream large reads, and use set-based bulk `UPDATE`/`DELETE` for mass changes — remembering they bypass the persistence context and caches. Loading everything as entities does not scale; these tools do.

MODULE 7 · QUERYING

CHAPTER 24

JPQL & HQL

Querying entities with an object-oriented query language instead of raw SQL.

JPQL (and Hibernate's superset, **HQL**) is an object-oriented query language: you query entities and their fields, not tables and columns. It looks like SQL but operates on your object model, and Hibernate translates it to the right SQL for your database.

Querying entities

JPQL references entity names and field names, and navigates associations with a dot — no join syntax needed for a simple path.

java

```
List<Order> orders = em.createQuery("""
    select o from Order o
    where o.status = :status and o.total > :min
    order by o.createdAt desc
    """, Order.class)
    .setParameter("status", Status.PAID)
    .setParameter("min", 500)
    .getResultList();
```

Always use bind parameters

WATCH OUT

Never build a query by concatenating user input into the string — that is a SQL-injection vulnerability. Always use named parameters (`:status`) or positional ones, which Hibernate binds safely.

Joins and aggregation

java

```
// explicit join and aggregate
List<Object[]> rows = em.createQuery("""
    select o.customer.name, count(o), sum(o.total)
    from Order o
    group by o.customer.name
    having sum(o.total) > 10000
    """).getResultList();
```

JPQL essentials

- Query by **entity and field** names, not tables and columns.
- Navigate associations with dots: `o.customer.name`.
- Use `join fetch` to load associations eagerly per query.

© 2026 ByteMart Digital

- HQL adds Hibernate-specific features on top of standard JPQL.

Query objects, not tables

JPQL and HQL let you query your entity model in a portable, object-oriented way, with Hibernate generating the SQL. Always bind parameters, navigate with dots, and use it as your default querying tool.

© 2026 ByteMart Digital

MODULE 7 · QUERYING

CHAPTER 25

The Criteria API

Building queries programmatically and type-safely — ideal for dynamic filtering.

String-based JPQL is concise but awkward when a query must be built dynamically from optional filters. The **Criteria API** builds queries programmatically and type-safely — the right tool for search screens where the query shape depends on which filters the user supplied.

A criteria query

```java
CriteriaBuilder cb = em.getCriteriaBuilder();
CriteriaQuery<Product> q = cb.createQuery(Product.class);
Root<Product> p = q.from(Product.class);

q.select(p).where(cb.greaterThan(p.get("price"), 100));
List<Product> result = em.createQuery(q).getResultList();
```

Dynamic predicates

The real strength: assemble a list of predicates conditionally, then combine them. Filters that were not supplied simply are not added.

```java
List<Predicate> filters = new ArrayList<>();
if (name != null)  filters.add(cb.like(p.get("name"), "%"+name+"%"));
if (min != null)   filters.add(cb.ge(p.get("price"), min));
if (active != null) filters.add(cb.equal(p.get("active"), active));

q.where(cb.and(filters.toArray(new Predicate[0])));
```

Criteria vs JPQL

	JPQL	Criteria
Form	string	programmatic
Best for	fixed queries	dynamic queries
Type safety	runtime	compile-time (with metamodel)

The metamodel

For full type safety, use the generated **static metamodel** (`Product_.price`) instead of string attribute names — so a renamed field breaks the build, not production.

© 2026 ByteMart Digital

Build queries in code

The Criteria API constructs queries programmatically and type-safely, excelling at dynamic filtering where optional criteria change the query shape. Use JPQL for fixed queries and Criteria for ones assembled at runtime.

MODULE 7 · QUERYING

CHAPTER 26

Native Queries & Projections

Dropping to raw SQL when needed, and selecting only the columns you require.

Sometimes you need database-specific SQL, a complex query JPQL cannot express, or just a few columns rather than whole entities. **Native queries** and **projections** handle these — letting you go straight to SQL and fetch exactly the data you need.

Native SQL queries

A native query runs raw SQL. Map the result to an entity, or to a projection when you only need some columns.

java

```
List<Product> hits = em.createNativeQuery(
    "SELECT * FROM product WHERE to_tsvector(name) @@ to_tsquery(?1)",
    Product.class)
    .setParameter(1, term)
    .getResultList();
```

Projections — fetch only what you need

Loading full entities to display three fields is wasteful. A **projection** selects just those columns into a DTO, avoiding the overhead of managed entities.

java

```
// JPQL constructor expression into a DTO
List<ProductSummary> rows = em.createQuery("""
    select new com.bytemart.ProductSummary(p.id, p.name, p.price)
    from Product p
    """, ProductSummary.class).getResultList();
```

When to use native queries

- Database-specific features (full-text search, window functions).
- Complex reporting queries JPQL cannot express cleanly.
- Performance-critical statements you want to hand-tune.
- Bulk operations against specific SQL syntax.

WATCH OUT

Native queries tie you to a specific database and bypass Hibernate's caches and dirty checking. Use them where they earn their keep, but prefer JPQL for portability and keep native SQL to the genuine exceptions.

© 2026 ByteMart Digital

Raw SQL and lean results

Native queries give you full SQL power for database-specific or complex statements, and projections fetch only the columns you need into DTOs. Reach for JPQL first, native SQL when necessary, and projections whenever you do not need full entities.

© 2026 ByteMart Digital

MODULE 7 · QUERYING

CHAPTER 27

Named Queries & DTO Projections

Pre-defining and validating queries, and returning tailored DTOs for read models.

Scattering query strings through your code makes them hard to find and easy to break. **Named queries** centralise and pre-validate them, and **DTO projections** return purpose-built read models — together giving you organised, efficient reads.

Named queries

A `@NamedQuery` defines a JPQL query on the entity, given a name. Hibernate validates it at startup, so a broken query fails fast rather than at runtime.

java

```java
@Entity
@NamedQuery(name = "Order.paidSince",
    query = "select o from Order o where o.status = 'PAID' " +
            "and o.createdAt >= :since")
public class Order { }

// used by name
em.createNamedQuery("Order.paidSince", Order.class)
  .setParameter("since", cutoff).getResultList();
```

DTO projections for read models

For read-heavy screens, return a **DTO** shaped for the view rather than the entity. This fetches only needed columns, avoids lazy-loading surprises, and decouples the read model from the entity.

java

```java
public record OrderRow(Long id, String customer, int total) { }

em.createQuery("""
    select new com.bytemart.OrderRow(o.id, o.customer.name, o.total)
    from Order o
    """, OrderRow.class).getResultList();
```

Benefits

- Named queries are **validated at startup** and easy to locate.
- DTO projections fetch **only needed columns** — faster reads.
- Read models are **decoupled** from the entity's shape.
- No managed entities means no accidental dirty updates.

© 2026 ByteMart Digital

Organised queries, tailored results

Named queries centralise and pre-validate your JPQL, and DTO projections return lean read models shaped for each view. Together they keep queries maintainable and reads efficient — the professional way to handle a read-heavy application.

© 2026 ByteMart Digital

MODULE 8 · CACHING, CONCURRENCY & TUNING

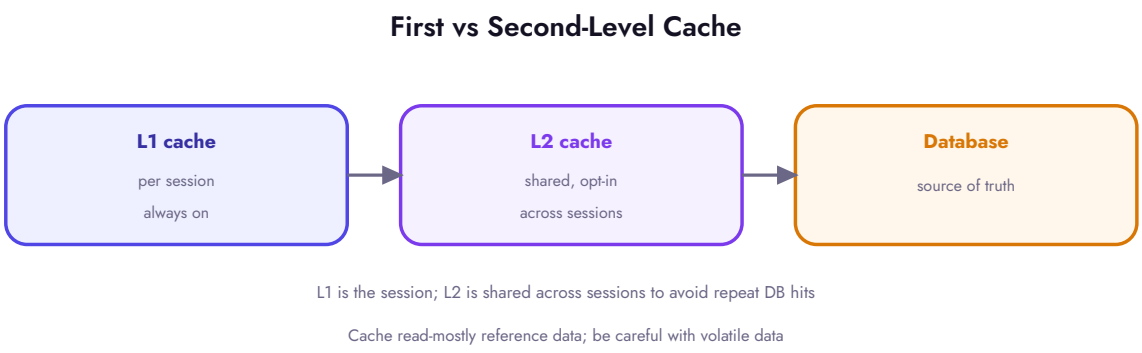
CHAPTER 28

The Second-Level Cache

Caching entities across sessions to cut database load — and where caching goes wrong.

The first-level cache lives only for one session. The **second-level cache** is shared across sessions, letting Hibernate serve frequently-read entities without hitting the database at all. Used on the right data it is a big win; used carelessly it serves stale results.

L1 vs L2



L1 is per-session and always on; L2 is shared across sessions and opt-in.

Cache	Scope
First-level (L1)	one session, always on
Second-level (L2)	shared, opt-in
Query cache	caches query result ids

Enabling L2 caching

L2 caching is opt-in per entity, backed by a provider like Ehcache or Infinispan. Mark cacheable entities and choose a concurrency strategy.

```java
@Entity
@Cacheable
@Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
public class Country {   // reference data — read often, changes rarely
    @Id String code;
    String name;
}
```

What to cache

- **Reference data** — countries, categories, config that rarely changes.

- **Read-mostly** entities with high read-to-write ratios.
- Data where slightly stale reads are acceptable.
- Not frequently-updated or strongly-consistent data.

WATCH OUT

Caching volatile or externally-modified data is dangerous — the cache serves stale values, and changes made outside Hibernate are invisible to it. Cache read-mostly reference data, and pick a concurrency strategy that matches your consistency needs.

Cache read-mostly data, carefully

The second-level cache shares entities across sessions to cut database load, but only helps for read-mostly reference data. Enable it selectively, choose the right concurrency strategy, and never cache volatile data you cannot afford to see stale.

MODULE 8 · CACHING, CONCURRENCY & TUNING

CHAPTER 29

Transactions & Locking

Ensuring correctness under concurrency with optimistic and pessimistic locking.

When two transactions touch the same data at once, one can silently overwrite the other's changes — the **lost update** problem. Hibernate offers two locking strategies, **optimistic** and **pessimistic**, to keep concurrent access correct.

Optimistic locking

Optimistic locking assumes conflicts are rare. A `@Version` column is checked on every update; if another transaction changed the row first, the version no longer matches and Hibernate throws — no database lock held.



The version column detects concurrent edits without holding a DB lock

A version column detects concurrent edits at update time, without holding a lock.

```java
@Entity
public class Account {
    @Id @GeneratedValue Long id;
    @Version
    private long version;      // checked and incremented on each update
    private int balance;
}
// concurrent update -> OptimisticLockException on the stale one
```

Pessimistic locking

Pessimistic locking assumes conflicts are likely. It acquires a database lock when reading, blocking others until the transaction completes — stronger, but it holds locks and can reduce throughput or deadlock.

```java
Account a = em.find(Account.class, id,
    LockModeType.PESSIMISTIC_WRITE);   // locks the row now
```

Choosing a strategy

	Optimistic	Pessimistic
Assumes	conflicts rare	conflicts likely
Cost	retry on conflict	holds a DB lock
Default choice	yes	only when needed

TIP

Prefer optimistic locking with a `@Version` field as your default — it scales well and adds no locks. Reserve pessimistic locking for genuinely contended resources like inventory counters or account balances under heavy concurrent writes.

Guard against lost updates

Concurrent writes can silently overwrite each other; a `@Version` field (optimistic locking) detects this cheaply and is the right default, while pessimistic locking holds a real lock for highly-contended data. Choose by how likely conflicts are.

© 2026 ByteMart Digital

MODULE 8 · CACHING, CONCURRENCY & TUNING

CHAPTER 30

Performance Tuning & Best Practices

Diagnosing and fixing Hibernate performance, and the habits that keep the data layer healthy.

Hibernate is fast when used well and slow when used carelessly. Most performance problems come from a handful of recurring mistakes — and once you know them, they are easy to avoid. This closing chapter gathers the diagnostics and habits that keep the data layer healthy.

Diagnose before you tune

- Enable **SQL logging** and Hibernate **statistics** in development.
- Count the queries per request — a burst signals N+1.
- Watch for **SELECT *** loading entities where a projection would do.
- Profile the actual database, not just the Java side.

The recurring performance sins

- **N+1 queries** from lazy associations iterated in a loop.
- **Eager** `@ManyToOne` loading unneeded data on every fetch.
- Loading **full entities** for read-only screens instead of DTOs.
- **IDENTITY** keys disabling batch inserts.
- Huge persistence contexts in long bulk transactions.

Best-practice habits

- Make associations **lazy**; fetch precisely per query.
- Use **projections / DTOs** for reads that do not need entities.
- Prefer **SEQUENCE** ids and enable JDBC batching for writes.
- Default to **optimistic locking** with `@Version`.
- Let a **migration tool** own the schema; Hibernate validates it.

Design the mapping for the queries

The overarching principle: design your mappings and fetch strategies around how the application actually reads and writes data. Hibernate rewards a mapping that fits its access patterns and punishes one that fights them.

> **Measure, then fix the usual suspects**
> Most Hibernate slowness is N+1, eager loading, or over-fetching — all visible the moment you enable SQL logging. Make associations lazy, fetch precisely, use DTOs for reads, batch your writes, and design mappings around real access patterns.

© 2026 ByteMart Digital